

Mode Opérateur

Plateforme Streamlit OPSAM

Alimentation · Optimisation · Diffusion sécurisée

DREAL Bourgogne-Franche-Comté — Juillet 2026

Ce document décrit les étapes complètes pour préparer les données OPSAM, optimiser le bundle, configurer un territoire, déployer l'application sur Internet avec un lien sécurisé, et dupliquer la plateforme pour d'autres projets.

#	Section	Objectif
1	Architecture & prérequis	Vue d'ensemble et installation
2	Préparer les données	Exécuter le pipeline OPSAM
3	Configurer le territoire	Fichier config/territoire.yaml
4	Optimiser pour le déploiement	Réduire ~99 Mo → ~25 Mo
5	Protéger par mot de passe	Hash SHA-256 + st.secrets
6	Créer le dépôt GitHub privé	Versionner code + données
7	Déployer sur Streamlit Cloud	Mise en ligne gratuite
8	Partager le lien aux partenaires	Lien privé sécurisé
9	Mettre à jour les données	Nouveau modèle OPSAM
10	Dupliquer pour un autre territoire	Nouveau projet en ~1 heure

1. Architecture & Prérequis

La plateforme repose sur une architecture en trois couches :

1 — Sources NAS	Sorties OPSAM (synthèse M1/M2, matrice VL, réseau routier, zonage, lookup EPCI)
↓ <i>scripts/prepare_*.py</i>	
2 — Bundle data/	Fichiers précalculés (CSV, GeoJSON, Parquet, GPKG) — ~25 Mo après optimisation
↓ <i>GitHub + Streamlit Cloud</i>	
3 — Application web	app.py (Streamlit) — 6 pages d'analyse, export CSV/PDF, login sécurisé

Prérequis logiciels

Outil	Version	Installation
Python	3.11 ou 3.13	python.org/downloads
Bibliothèques Python	voir requirements.txt	pip install -r requirements.txt
Git	≥ 2.40	git-scm.com
Compte GitHub	gratuit	github.com (dépôt privé)
Compte Streamlit Cloud	gratuit	share.streamlit.io

Installer toutes les dépendances depuis le dossier du projet :

```
cd D:\12__OPSAM\plateforme_analyse_inter_SERM
python -m pip install -r requirements.txt
```

Structure du projet

Fichier / Dossier	Rôle
app.py	Point d'entrée Streamlit — 6 pages d'analyse
data_loader.py	Couche de lecture des données (CSV, GeoJSON, Parquet)
visualizations.py	Fabrique de graphiques Plotly (cartes, Sankey, heatmaps...)
datagouv_context.py	Page 5 — données contextuelles INSEE / SNCF
pdf_export.py	Export PDF des rapports (ReportLab + Kaleido)
auth.py	Module de login par mot de passe
config/territoire.yaml	■ Configuration du territoire (noms, codes, couleurs...)
config/territoire_loader.py	Chargement et validation du YAML
data/	Bundle de données précalculées (~25 Mo après optimisation)
scripts/	Pipeline de préparation et d'optimisation des données

.streamlit/secrets.toml	Secrets (mot de passe, token Mapbox) — ne pas versionner
-------------------------	--

2. Préparer les données

Les données sont précalculées depuis les sorties brutes du modèle OPSAM stockées sur le NAS. Le pipeline est orchestré par un script maître.

2.1 Vérifier l'accès aux sources NAS

Les scripts ont besoin d'accéder aux partages réseau suivants (configurables dans **config/territoire.yaml**, section *chemins*) :

Partage réseau	Contenu
\\poste-2-170\OPSAM_v3\OPSAM\Outputs\Ref2024	Fichiers synthèse M1/M2
\\poste-2-170\OPSAM_v3\OPSAM\Base\Ref2024	Réseau routier shapefile
\\nas-bfc\COMMUN\21_MOBILITE\...	Lookup EPCI, zonage, EPCI shapefile

2.2 Lancer le pipeline complet (rebuild)

Ce script orchestre les 5 scripts de préparation dans l'ordre correct :

```
cd D:\12__OPSAM\plateforme_analyse_inter_SERM
python scripts/rebuild_for_new_perimeter.py
# Options utiles :
# --scenario Ref2025      (pour un nouveau scenario OPSAM)
# --export                (génère aussi EXPORT_LIVRABLES/)
# --verbose               (affiche le détail des étapes)
```

Le pipeline appelle dans l'ordre :

- ① prepare_synthese_serm.py — synthèse VL/PL par SERM et type de voie
- ② prepare_perimetres_serm.py — périmètres et zones OPSAM en GeoJSON
- ③ prepare_reseau_serm.py — réseau routier découpé par SERM (GPKG)
- ④ prepare_od_vl.py — matrices OD VL, top flux, échanges EPCI (Parquet)
- ⑤ prepare_limites.py — limites communes et EPCI par SERM (GeoJSON)

2.3 Vérifier le bundle produit

```
# Lister les fichiers produits dans data/
dir D:\12__OPSAM\plateforme_analyse_inter_SERM\data
```

Les fichiers attendus dans data/ :

Fichier	Description	Format
synthese_serm_vl_pl.csv	VKM par SERM / type voie / flux / distance	CSV ; ~25 Ko
matrice_inter_serm.csv	Matrice OD 4x4 entre zones	CSV ; <1 Ko
perimetres_serm.geojson	Polygones des périmètres SERM	GeoJSON ; ~150 Ko
zones_serm.geojson	Maillage fin OPSAM par SERM	GeoJSON ; ~2,5 Mo
limites_epci.geojson	Contours EPCI région	GeoJSON ; ~19 Mo brut
limites_communes_serm{N}.geojson	Contours communes par SERM	GeoJSON ; ~10 Mo brut
reseau_serm/{slug}.gpkg	Réseau routier avec trafics	GPKG ; ~67 Mo brut
top_od_vl_par_serm.parquet	Top N flux OD VL par SERM	Parquet ; ~100 Ko

top_od_com_serm.parquet	Top flux à la commune/EPCI	Parquet ; ~90 Ko
echanges_epci_serm.parquet	Échanges EPCI×EPCI par SERM	Parquet ; ~37 Ko
centroides_zones.parquet	Coordonnées centroïdes zones OPSAM	Parquet ; ~320 Ko

3. Configurer le territoire

Le fichier **config/territoire.yaml** est la source de vérité unique. Il contient tous les paramètres propres au territoire : noms, codes, couleurs, chemins NAS, départements de la région. **Ne jamais modifier SERM_INFO dans data_loader.py directement.**

3.1 Éditer config/territoire.yaml

Les sections à adapter pour chaque nouveau projet :

Section YAML	Paramètre	Exemple BFC	À modifier pour
plateforme	nom	Plateforme inter-SERM	Nom affiché dans l'UI
plateforme	institution	DREAL BFC	Organisation productrice
plateforme	scenario_opsam	Ref2024	Identifiant scénario OPSAM
region	nom / nom_court	Bourgogne-Franche-Comté / BFC	Région d'étude
region	departements	["21","25",...]	Codes INSEE départements
region	carte_centre_lat/lon	47.2 / 5.0	Centre carte par défaut
zones[]	code, nom, slug	1, SERM Dijonnais, dijon	Chaque territoire d'étude
zones[]	couleur	#C62828	Couleur hexadécimale carte
zones[]	macrozone	{source: M1, code: 1}	Lien avec sorties OPSAM M1/M2
zone_hors_serm	nom, code	Reste BFC (hors SERM), 0	Zone résiduelle (null si absent)
chemins	opsam_outputs / ...	\\poste-2-170\...	Chemins accès NAS

■ Les chemins NAS peuvent aussi être fournis via des variables d'environnement (SERM_OPSAM_DIR, SERM_LOOKUP_CSV, SERM_ZONAGE_SHP, SERM_EPCI_SHP) pour éviter de stocker des chemins réseau dans le YAML versionné.

3.2 Vérifier le chargement de la configuration

```
python -c "  
import sys; sys.path.insert(0, '.')  
from config.territoire_loader import cfg, codes_zones  
c = cfg()  
print('Territoire :', c.region.nom)  
print('Zones      :', codes_zones())  
"
```

Résultat attendu :

```
Territoire : Bourgogne-Franche-Comté  
Zones      : [1, 2, 3]
```

4. Optimiser les données pour le déploiement

Le bundle brut pèse ~99 Mo (sans le cache data.gouv). Le script d'optimisation le réduit à ~25 Mo en trois actions, sans perte visible à l'écran :

- Convertit les GPKG réseau en GeoParquet (géométries simplifiées à 30 m) : 67 Mo → ~15 Mo
- Simplifie les GeoJSON limites communes/EPCI (tolérance 30 m) : 29 Mo → ~5 Mo
- Déplace le cache data.gouv hors du dossier déployable (32 Mo exclus)

4.1 Lancer l'optimisation

```
cd D:\12__OPSAM\plateforme_analyse_inter_SERM
python scripts/optimize_for_deployment.py
# Les originaux sont conservés dans data/.backup_avant_optim/
# (ce dossier est exclu du dépôt Git par .gitignore)
```

4.2 Vérifier la taille finale

```
dir /s data | findstr /C:"octets"
```

■ Cible : dossier data/ < 30 Mo. Si la cible n'est pas atteinte, vérifier que les GPKG ont bien été convertis en .parquet et que les fichiers cache data.gouv ont été déplacés dans .backup_avant_optim/.

5. Protéger la plateforme par mot de passe

L'accès à la plateforme est protégé par un formulaire de connexion. Le mot de passe est stocké sous forme de hash SHA-256 dans les secrets Streamlit (jamais en clair dans le code).

5.1 Générer le hash du mot de passe

```
# Remplacer VOTRE_MOT_DE_PASSE par le mot de passe choisi
python -c "import hashlib; print(hashlib.sha256(b'VOTRE_MOT_DE_PASSE').hexdigest())"
```

Exemple de résultat (chaîne de 64 caractères hexadécimaux) :

```
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855
```

■ Conserver le mot de passe en clair dans un gestionnaire de mots de passe (KeePass, Bitwarden...). Le hash seul ne permet pas de retrouver le mot de passe.

5.2 Créer le fichier secrets.toml local (facultatif, pour tests)

Pour tester la protection en local, créer .streamlit/secrets.toml (ce fichier est dans le .gitignore, il ne sera jamais envoyé sur GitHub) :

```
# .streamlit/secrets.toml
MOT_DE_PASSE_HASH = "LE_HASH_GENERE_CI-DESSUS"
MAPBOX_TOKEN = "" # optionnel
```

5.3 Tester en local

```
streamlit run app.py
```

La page de connexion doit s'afficher. Saisir le mot de passe pour accéder à l'application.

6. Créer le dépôt GitHub privé

Le code et les données optimisées (~25 Mo) sont versionnés dans un dépôt GitHub privé. Streamlit Community Cloud se connecte directement à ce dépôt.

6.1 Créer le dépôt sur github.com

Se connecter sur github.com → **New repository** :

- Nom : plateforme-inter-serm (ou autre)
- Visibilité : **Private** (obligatoire)
- Ne pas initialiser avec README (on pousse un projet existant)

6.2 Initialiser et pousser le projet

```
cd D:\12__OPSAM\plateforme_analyse_inter_SERM
git init
git branch -M main
git remote add origin https://github.com/VOTRE_LOGIN/plateforme-inter-serm.git
# Vérifier que .gitignore exclut bien les fichiers sensibles et lourds
git status
git add .
git commit -m "feat: plateforme inter-SERM – déploiement initial"
git push -u origin main
```

■ Vérifier AVANT le push que .gitignore exclut bien : .streamlit/secrets.toml, data/.backup_avant_optim/, data/reseau_serm/*.gpkg, EXPORT_LIVRABLES/, __pycache__/

7. Déployer sur Streamlit Community Cloud

Streamlit Community Cloud est le service d'hébergement gratuit de Streamlit. Il surveille le dépôt GitHub et redéploie automatiquement à chaque push.

7.1 Connexion et création de l'application

- ① Ouvrir **share.streamlit.io** → Se connecter avec GitHub
- ② Cliquer **New app**
- ③ Sélectionner le dépôt plateforme-inter-serm
- ④ Branche : main — Fichier principal : app.py
- ⑤ Cliquer **Advanced settings**

7.2 Configurer les secrets

Dans l'onglet **Secrets** des Advanced settings, coller :

```
MOT_DE_PASSE_HASH = "LE_HASH_GENERE_A_L_ETAPE_5"
MAPBOX_TOKEN = "" # laisser vide ou renseigner un jeton Mapbox
```

- ⑥ Cliquer **Deploy!** — le déploiement prend 3 à 7 minutes.

7.3 Résultat

L'application est accessible via une URL du type :

```
https://plateforme-inter-serm-abc123.streamlit.app
```

Limites de la formule gratuite

Ressource	Limite	Impact
RAM	1 GB	Suffisant pour les 3 SERM BFC après optimisation
CPU	1 vCPU partagé	Temps de chargement des cartes ~3-5 s
Stockage dépôt	< 1 GB	~25 Mo de données ✓ très large marge
Apps simultanées	3	Une par territoire
Inactivité	Veille après 7 j sans visite	Réveil automatique à la première visite

8. Partager le lien aux partenaires

La plateforme n'est pas indexée par les moteurs de recherche. L'accès nécessite de connaître à la fois l'URL et le mot de passe.

8.1 Communiquer l'accès

Envoyer par e-mail sécurisé (ou messagerie interne) :

- L'URL de l'application (<https://xxx.streamlit.app>)
- Le mot de passe d'accès
- Ce mode opératoire (optionnel)

8.2 Expérience de connexion partenaire

- ① Le partenaire ouvre l'URL dans un navigateur
- ② Une page de connexion s'affiche (titre, champ mot de passe)
- ③ Après connexion correcte, accès complet à la plateforme
- ④ La session reste active tant que l'onglet est ouvert
- ⑤ Le bouton « Se déconnecter » est disponible dans la sidebar

8.3 Changer le mot de passe

Générer un nouveau hash (étape 5.1), puis :

- Sur Streamlit Cloud : **Settings > Secrets** → mettre à jour `MOT_DE_PASSE_HASH`
- L'application se redémarre automatiquement

9. Mettre à jour les données

Quand un nouveau scénario OPSAM est disponible (ex. Ref2025), la mise à jour se fait en 4 commandes :

9.1 Mettre à jour le scénario dans la configuration

```
# Éditer config/territoire.yaml :  
#   scenario_opsam: "Ref2025"   (à la place de Ref2024)  
# Et mettre à jour le chemin si le dossier NAS a changé :  
#   chemins.opsam_outputs: '\\poste-2-170\\...\\Ref2025'
```

9.2 Régénérer le bundle depuis le NAS

```
python scripts/rebuild_for_new_perimeter.py --scenario Ref2025
```

9.3 Ré-optimiser pour le déploiement

```
python scripts/optimize_for_deployment.py
```

9.4 Pousser sur GitHub → déploiement automatique

```
git add data/  
git commit -m "data: mise à jour bundle OPSAM Ref2025"  
git push  
# Streamlit Cloud détecte le push et redéploie automatiquement (~3 min)
```

10. Dupliquer pour un autre territoire

La plateforme est conçue pour être réutilisable. Pour un nouveau projet (autre région, autre scénario), seuls le fichier de configuration et les données changent. Le code applicatif reste identique.

10.1 Copier le projet

```
# Option A : copier le dossier entier
xcopy D:\12__OPSAM\plateforme_analyse_inter_SERM
      D:\MON_PROJET\plateforme_mon_territoire /E /I
# Option B : cloner depuis GitHub
git clone https://github.com/VOTRE_LOGIN/plateforme-inter-serm
      plateforme-mon-territoire
```

10.2 Adapter config/territoire.yaml

C'est la **seule étape de paramétrage** nécessaire pour l'application. Modifier les sections :

- plateforme : nom, institution, scénario OPSAM
- region : nom, codes départements, centre carte
- zones : liste des territoires d'étude (N quelconque)
- chemins : chemins NAS vers les sorties OPSAM du nouveau projet

10.3 Préparer les données du nouveau territoire

```
# Depuis le dossier du nouveau projet :
python scripts/rebuild_for_new_perimeter.py
python scripts/optimize_for_deployment.py
```

10.4 Créer un nouveau dépôt GitHub privé et déployer

Répéter les étapes 6 et 7 pour ce nouveau projet.

Ce qui est commun à tous les déploiements

Élément	Commun ?	Note
Structure des pages Streamlit	■ Oui	Identique pour tous les territoires
Graphiques Plotly (Sankey, heatmap, lignes de désir...)	■ Oui	Dynamiques selon les données
Export CSV / PDF	■ Oui	Généré depuis les données chargées
Authentification mot de passe	■ Oui	Hash différent par déploiement
Format des données OPSAM (VKM, flux E/T/I, D1-D5)	■ Oui	Standard OPSAM
Noms, codes, couleurs des territoires	■■ Config	Via territoire.yaml
Chemins NAS sources	■■ Config	Via territoire.yaml ou env vars
Logique M1/M2 si >1 modèle OPSAM	■■ Partiel	À adapter si autre structure

